



SAPIENZA
UNIVERSITÀ DI ROMA

Transformer-Based Approaches to Rule Violation Detection in Online Communities

Faculty of Information Engineering, Informatics, and Statistics.

Corso di Laurea in Applied Computer Science and Artificial Intelligence

Candidate

Zhansaya Orazbay

ID number 2058167

A handwritten signature in black ink, appearing to be 'Zhansaya Orazbay'.

Thesis Advisor

Mattia Samory

A handwritten signature in black ink, appearing to be 'Mattia Samory'.

Academic Year 2025/2026

Thesis not yet defended

Transformer-Based Approaches to Rule Violation Detection in Online Communities

Bachelor's thesis. Sapienza – University of Rome

© 2026 Zhansaya Orazbay. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: orazbay.2058167@studenti.uniroma1.it

Abstract

Online discussion platforms generate large volumes of user-generated content that must be actively moderated to maintain community standards. Platforms such as Reddit and Lemmy rely on community-specific rules to regulate user behavior: rules that differ across communities and require contextual interpretation to apply consistently. As communities grow, manual rule enforcement becomes increasingly impractical. Moderators are typically unpaid volunteers, reviewing high volumes of content in real time with no automated assistance beyond broad toxicity filters.

Existing automated moderation systems treat the problem as toxicity classification: is this comment harmful or not? This frame of reference ignores the specific rule descriptions that moderators actually use to make decisions. A comment that violates a civility rule in one community may be perfectly acceptable in another. The relevant question is not whether a comment is toxic, but which rule it violates and whether a model can identify that rule from its textual description alone.

This thesis formulates rule violation detection as a rule selection task: given a user-generated comment and a set of community-specific rules written in natural language, identify which rule has been violated. Two transformer-based approaches are implemented and evaluated using *bert-base-uncased*: an extractive question answering formulation, in which the model predicts the span of text corresponding to the violated rule within a structured input sequence, and a multiple-choice formulation, in which each candidate rule is evaluated independently against the comment.

Both models are evaluated on three dataset splits designed to test different generalization conditions: in-distribution performance, generalization to previously unseen rule descriptions, and transfer to previously unseen communities. The multiple-choice formulation outperforms the extractive approach across all three splits, achieving a macro-F1 of 0.691 on the stratified split compared to 0.588 for the extractive model. On the unseen-communities split, statistical significance testing reveals that the two models make qualitatively different errors, suggesting that their combination could improve cross-community transfer.

The most striking result is on the unseen-rules split, where a Linear SVM baseline operating purely on comment text achieves 0.196 macro-F1, outperforming both transformer-based models. This suggests that standard fine-tuning teaches models to associate comment patterns with specific rule words rather than to reason about what a rule means. Generalizing to new rule descriptions remains an open problem that standard fine-tuning approaches cannot solve.

Contents

1	Introduction	1
1.1	Research scope	1
1.2	Thesis objectives	2
1.3	Structure of the thesis	2
2	Theoretical foundations	4
2.1	Rule-based moderation in online communities	4
2.2	Transformer-based language models	6
2.3	BERT	6
2.4	Extractive and multiple-choice question answering	7
3	Problem formulation and modeling approach	9
3.1	Task formulation	9
3.2	Dataset and splits	11
3.3	Input representation	13
3.4	Model architectures	13
3.4.1	Extractive question answering formulation	14
3.4.2	Multiple-choice formulation	15
4	Training setup	17
4.1	Training configuration	17
4.1.1	Model initialization and tokenization	17
4.1.2	Optimizer and training stability	19
4.1.3	Data loading and batching	20
4.1.4	Data augmentation	22
4.1.5	Computational setup	23
4.2	Evaluation metrics	24
4.2.1	Classification metrics	24

4.3	Baseline models	25
4.4	Hyperparameter selection	27
4.5	Data augmentation selection	28
4.6	Extractive question answering training configuration	30
5	Experiments and Evaluation	31
5.1	Multiple-choice model results	31
5.2	Extractive question answering model results	32
5.3	Comparison of modeling approaches	33
5.4	Statistical significance testing	34
6	Conclusion	36
6.1	Summary	36
6.2	Future work	38
6.2.1	Zero-shot and few-shot generalization to unseen rules	38
6.2.2	Ensemble of multiple-choice and extractive models	38
6.2.3	Stronger pretrained models	38
6.2.4	Deployment and real-world integration	39
	Bibliography	40
	Implementation Details	42
.1	Data Augmentation Implementations	42
.2	McNemar's Test Implementation	44
.3	Baseline Implementations	45

Chapter 1

Introduction

1.1 Research scope

This thesis addresses the problem of automatic rule violation detection in online communities. Given a user comment and a set of community-specific rules, the goal is to identify which rule was violated. Unlike previous approaches that frame moderation as binary toxicity classification [1], this formulation conditions the model’s prediction on the rule text itself. It makes it applicable to any community without predefined label categories. In practice, three properties of the problem make it difficult.

First, community rules written in natural language are ubiquitous across online platforms: virtually every community defines acceptable behavior through rules such as *Be civil* or *No off-topic posts*. However, despite their prevalence, these rules resist uniform interpretation. What counts as uncivil in a scientific debate community may be standard discourse elsewhere, and the same comment may be flagged in one community and ignored in another depending on local norms and discussion context. A rule description alone does not determine its application, therefore it requires reasoning over the relationship between the comment and the specific community context in which it appears.

Second, rule sets are not standardized across communities. Beyond differences in interpretation, communities differ in the number, wording, and scope of their rules. Some have two rules, others have eleven, and the same concept may be encoded under different phrasings. Unlike standard classification tasks with a fixed label set, the candidate rules here are part of the input and change from one community to the next. A model must therefore reason over whichever rule set it is given, rather than selecting from a predefined schema it was trained on.

Third, detecting a violation requires reasoning over the relationship between a comment and a specific rule description. By far the largest share of academic and commercial tools for automated moderation has focused on narrow linguistic categories of abuse at the level of the single comment: flagging slurs, threats, or explicit hostility [1]. A comment that contains no offensive language can still violate a rule about topic relevance, reposting behavior, or community-specific norms — cases where the linguistic surface of the comment alone is insufficient to determine whether a violation occurred. Identifying such violations requires reasoning over the comment and the rule description it is being evaluated against.

This thesis addresses these challenges by framing rule violation detection as a natural language processing task in which a transformer-based model receives a user comment together with a set of community rules and must determine which rule has been violated. The models are trained and evaluated under various hyperparameter configurations and data augmentation strategies to assess their robustness and generalization ability. Practical deployment and integration into real-world moderation systems remain outside the scope of this work.

1.2 Thesis objectives

The challenges described above motivate the following objectives:

- Formulate rule violation detection as a natural language processing problem that models the relationship between user comments and textual rule descriptions, rather than treating it as generic toxicity classification.
- Implement two transformer-based approaches for rule violation detection: an extractive question answering formulation and a multiple-choice formulation.
- Evaluate the impact of different hyperparameter configurations on model convergence and generalization performance across three evaluation splits.
- Investigate the effect of data augmentation strategies on model robustness under out-of-distribution conditions. Since community rules vary in number, wording, and order across communities, augmentation strategies — such as shuffling rule order, excluding non-violated rules, or perturbing numerical expressions — may reduce reliance on surface patterns and improve generalization to previously unseen rules and communities.
- Analyze the ability of both models to generalize to previously unseen community rules and previously unseen online communities, using dedicated dataset splits designed for each generalization condition.
- Compare the two modeling approaches statistical significance testing to assess whether observed performance are likely to reflect architectural differences rather than sampling variation.

1.3 Structure of the thesis

Chapter [Theoretical foundations](#) (p. 4) introduces the core concepts underlying this work: rule-based moderation in online communities, transformer-based language models, the BERT architecture, and the extractive and multiple-choice question answering formulations that motivate the modeling choices made in subsequent chapters.

Chapter [Problem formulation and modeling approach](#) (p. 9) defines the task formally, describes the dataset collected from Lemmy and its three evaluation splits, explains the input representation used for each formulation, and presents the two model architectures in detail.

Chapter **Training setup** (p. 17) describes the full experimental configuration: model initialization, tokenization, optimizer settings, data loading and batching, data augmentation strategies, evaluation metrics, baseline models, hyperparameter selection, and augmentation selection experiments.

Chapter **Experiments and Evaluation** (p. 31) presents the results for both models across all three evaluation splits, compares their performance against the strongest baseline, and reports the results of McNemar’s statistical significance testing.

Chapter **Conclusion** (p. 36) summarizes the main findings, acknowledges the limitations of the work, and outlines four directions for future research.

Chapter 2

Theoretical foundations

This chapter introduces the background knowledge necessary to situate the modeling choices made in this thesis. It proceeds in four parts. Section 2.1 describes the context of rule-based moderation in online communities, surveys the relevant social computing and automated moderation literature, and motivates the need for rule-aware approaches. Section 2.2 introduces transformer-based language models and explains why their ability to jointly encode multiple text segments makes them well-suited for reasoning over comments and rule descriptions. Section 2.3 describes the BERT architecture specifically, including the pretraining objectives and input representations that are directly relevant to this thesis. Finally, Section 2.4 introduces the extractive and multiple-choice question answering formulations that serve as the basis for both modeling approaches explored here.

2.1 Rule-based moderation in online communities

Online discussion platforms rely on community-specific rules to regulate user behavior. Unlike platform-wide content policies, which apply uniformly across all communities, community rules are locally defined, vary substantially in scope and wording, and require contextual interpretation to apply consistently. The text of a comment alone is rarely sufficient to determine whether a rule has been violated: moderators must also weigh social context, community norms, and prior judgment calls that leave no trace in the words themselves. This thesis focuses on the linguistic dimension of the problem: given the text of a comment and the textual descriptions of community rules, can a model identify which rule was violated? While this framing does not capture the full complexity of moderation in practice, it represents a tractable and meaningful subproblem with direct practical relevance.

At scale, this reasoning process cannot be performed manually. It is useful here to distinguish between two modes of moderation. Centralized moderation, as practiced by platform operators, applies uniform policies across all communities and is typically enforced by paid staff or automated systems [2]. Community moderation, by contrast, is carried out by volunteers who enforce locally-defined rules that vary substantially between communities in their number, wording, and scope [3, 4]. The burden this places on volunteers is substantial. Reddit moderators alone worked an estimated minimum of 466 hours per day in 2020, which worth more than three million dollars annually at market rates [5] and grows with every new community and every new

user.

This decentralized structure has a direct consequence for automated moderation: rather than a single universal rule set, the moderation landscape consists of thousands of locally-defined rule systems. Each rule reflects the specific norms and expectations of its community. Chandrasekharan et al. demonstrated this through analysis of rule structures across Reddit communities, showing that norms and their enforcement vary substantially even for superficially similar rules [6]. This heterogeneity reflects that different communities have different values and that rule enforcement requires community-level understanding rather than platform-wide uniformity [3].

The importance of rule-specific reasoning is also reflected in how moderators actually behave. Jhaver et al. found that moderators on Reddit rely heavily on rule citations when removing content and that users who receive rule-specific removal explanations are significantly less likely to reoffend [7]. This means an automated system that identifies which rule was violated — rather than assigning a generic toxicity label — is both more actionable and more aligned with how human moderators actually work.

The tools available to moderators provide limited support for rule-based decision-making. The moderation queue — the primary interface through which moderators review flagged content on platforms like Reddit — presents comments for human judgment without automated assistance beyond basic toxicity filters [8]. Research on how moderators actually use this interface reveals that moderation practices vary widely across communities and that moderators frequently leave the queue to consult community rules, moderation logs, and prior decisions before acting [8, 9]. This suggests a gap in current moderation tooling: moderators must manually reason about the relationship between a flagged comment and the specific rules of their community, with no automated system to assist in identifying which rule applies. The automated moderation literature has begun moving in this direction, as discussed below, but the specific formulation of open-ended rule selection over dynamically provided rule text remains underexplored.

Existing automated moderation systems have largely focused on classification of toxicity [1, 10], assigning labels based only on comment content without consulting the specific rules of each community. However, a smaller body of work has moved toward rule-aware approaches. Park et al. constructed NormVio, a Reddit dataset covering a broader range of violations than toxicity alone — spam, off-topic posts, formatting issues — and found that including community rules as model input improves detection [11]. Building on this, He et al. proposed CPL-NoViD, a prompt-based approach that extends NormVio to predict the specific rule being violated rather than just the violation category [12]. Most recently, Xin et al. proposed a framework that directly integrates community rule descriptions into moderation decisions, demonstrating that incorporating rules substantially improves performance over rule-agnostic baselines [13]. The present thesis shares this motivation, but rather than treating rule integration as a binary moderation decision, the task is framed as selecting which specific rule was violated from a dynamically provided candidate set.

2.2 Transformer-based language models

Rule violation detection requires modeling the relationship between two pieces of text: a user comment and a rule description. Traditional NLP approaches handle this poorly. Bag-of-words representations treat texts as unordered collections of words, discarding syntactic structure and making it impossible to capture interactions between the comment and the rule. Static word embeddings such as Word2Vec [14] or GloVe [15] assign a fixed vector to each word regardless of context – the word *promotion* has the same representation in a discussion about career advancement and in a community rule that bans advertising.

Transformer-based language models address both limitations. Through self-attention, they process the entire input at once, so each token’s representation is shaped by all other tokens simultaneously, including those in a different text segment. This means that when a comment and a rule description are encoded together, the model can attend to the relationship between specific words in the comment and specific words in the rule. This architecture was introduced by Vaswani et al. (2017) [16] and became the foundation for most modern language understanding systems. Unlike recurrent neural networks such as LSTMs [17], which process text sequentially and struggle to capture dependencies between distant tokens, transformers process the entire sequence in parallel.

Modern transformer systems follow a two-stage training procedure. During pre-training, the model learns general linguistic patterns from large text corpora: how words relate to each other, how sentences are structured, and how meaning changes with context. During fine-tuning, the pretrained model is adapted to a specific downstream task using a relatively small labeled dataset [18]. This strategy has been shown to produce strong results in a wide range of NLP tasks with minimal architectural changes to the task [19].

In this thesis, the ability of transformers to jointly encode a comment and a set of rule descriptions within a single input sequence is the central property being exploited. The model must attend to the relationship between specific words in the comment and specific words in the rule text. It is a form of cross-segment reasoning that bag-of-words and static embedding approaches cannot support.

2.3 BERT

BERT (Bidirectional Encoder Representations of Transformers) is a transformer-based language model that processes text using bidirectional self-attention [18]. Earlier language models such as ELMo [20] used bidirectional LSTMs to produce contextual representations, but processed the two directions independently and concatenated the results. GPT [21] used a unidirectional transformer that reads text strictly left-to-right. BERT was the first model to apply deep bidirectional self-attention, considering both preceding and following contexts for each token simultaneously within a single unified architecture.

Two architectural properties make BERT particularly relevant for the detection of rule violations. First, BERT can process multiple text segments within a single input sequence by separating them with a special *[SEP]* token. This allows the model to encode a comment and a rule description together, capturing interactions between them within a shared representation space. Second, BERT’s pretraining

procedure – masked language modeling and next sentence prediction – gives the model a strong prior for reasoning about relationships between sentence pairs, which transfers naturally to the comment-rule matching task.

Transformer-based models have shown strong results on tasks closely related to rule violation detection. In toxicity detection, BERT-based classifiers substantially outperform bag-of-words and LSTM-based approaches, particularly in tasks that require sensitivity to context and meaning [10]. The NormVio dataset introduced by Park et al. demonstrated that BERT-based models incorporating community context outperform context-insensitive baselines by nearly 50% in F1 [11]. The same ability to reason over pairs of texts has made transformers the go-to architecture for question answering, textual inference, and semantic similarity tasks [19]. All of these are structurally similar to the comment-rule matching problem in this thesis.

BERT’s flexibility in handling multi-segment inputs and its strong performance on text-pair tasks make it a natural backbone for both modeling approaches explored in this thesis.

2.4 Extractive and multiple-choice question answering

Rule violation detection requires identifying the specific rule that a comment violates. It is a prediction that depends on the textual content of both the comment and the rule descriptions. This structure suggests a natural analogy to reading comprehension: rather than selecting from a fixed set of predefined labels, the model must reason over the actual text of the rules and locate the relevant one. This thesis explores two question answering formulations that make this analogy concrete.

In extractive question answering, the answer is a span of text within the input context. The model predicts the start and end token positions of that span rather than selecting from a fixed label set. Extractive QA is standardly evaluated on datasets such as SQuAD [22] and its adversarial variant SQuAD 2.0 [23]. It adds unanswerable questions to test whether models can abstain when no valid span exists.

In this thesis, the extractive formulation is applied over the rule list: all candidate rules are concatenated into a structured sequence using custom boundary tokens, and the model predicts the span corresponding to the violated rule within that sequence. This formulation supports variable-length rule sets. The model does not need to know in advance how many rules exist, because it operates over their text directly.

In multiple-choice question answering, each candidate answer is presented as a separate option paired with the question and context. The model evaluates all options and selects the one that best matches the input. Multiple-choice QA is studied on datasets such as RACE [24] and HellaSwag [25], which require common-sense reasoning over multiple textual alternatives.

In this thesis, each community rule is treated as a candidate option paired independently with the comment. The model assigns a relevance score to each comment-rule pair and selects the rule with the highest score after softmax normalization.

The two formulations differ in a structurally important way. The extractive model must identify the correct rule span within a long sequence containing all candidate rules – its attention is split across the entire rule list. The multiple-choice model

evaluates each comment-rule pair independently, which allows focused comparison between the comment and a single rule at a time. This structural difference turns out to matter for performance, as discussed in Chapter [Experiments and Evaluation](#).

Chapter 3

Problem formulation and modeling approach

This chapter defines the task addressed in this thesis and describes the two modeling approaches explored. Section 3.1 introduces the task formulation as a rule selection problem and defines the formal notation used throughout. Section 3.2 describes the dataset and the three evaluation splits designed to assess generalization under different conditions. Section 3.3 explains how raw instances are converted into model inputs for each formulation. Section 3.4 describes the two model architectures — extractive question answering and multiple-choice — and their prediction mechanisms.

3.1 Task formulation

The task addressed in this work is the automatic identification of a violated community rule given a user-generated comment and a corresponding set of moderation rules. Formally, the problem can be defined as a rule selection task conditioned on the textual context.

The most common formulation of automated content moderation is moderation action prediction: given a comment, decide whether to approve or remove it [11]. This thesis adopts a more fine-grained formulation. Rather than predicting a binary outcome, the model must identify which specific rule from the community’s rule set was violated. This prediction requires reasoning over the textual content of both the comment and the rules.

Each instance in the dataset consists of a comment posted by a user and a set of rules associated with a specific online community. The objective of the model is to determine which rule of the provided set has been violated by the comment. Let a comment be denoted as c , and the corresponding set of community rules as

$$R = \{r_1, r_2, \dots, r_n\}$$

One rule in every set is a special *Safe* label, representing the case where no community rule was violated and the comment was removed for other reasons or not removed at all. Rather than treating this as a separate binary decision, the *Safe* label is

included as a candidate rule within R , allowing the model to handle both violated and non-violated cases within a single unified formulation.

The goal is to predict the index i such that rule $r_i \in R$ represents the violated rule:

$$f(c, R) \rightarrow r_i$$

To investigate different modeling strategies for this task, two alternative formulations are considered in this thesis: extractive question answering and multiple-choice question answering.

In the extractive formulation, the rule set is represented as a structured textual context concatenated with the input comment. This formulation treats rule violation detection as a span prediction problem over structured rule representations [18, 22].

Formally, the model predicts the start and end token positions of the violated rule span by computing logit scores over all token positions:

$$p_k^{\text{start}} = \frac{\exp(w_{\text{start}}^\top h_k)}{\sum_{j=1}^L \exp(w_{\text{start}}^\top h_j)} \quad (3.1)$$

$$p_k^{\text{end}} = \frac{\exp(w_{\text{end}}^\top h_k)}{\sum_{j=1}^L \exp(w_{\text{end}}^\top h_j)} \quad (3.2)$$

Here k denotes the candidate position being scored, j ranges over all positions in the input sequence to form the normalizing denominator, h_k is the contextual token representation produced by BERT at position k , and w_{start} , w_{end} are learned projection vectors.

The predicted positions $\hat{k}^{\text{start}}$ and $\hat{k}^{\text{end}}$ are then mapped to a rule identifier. The predicted positions do not always align neatly with rule boundaries: the span may land inside a single rule, straddle a boundary, or cover parts of multiple rules. To handle this, each rule in the input sequence is associated with a set of predefined token boundaries, and the predicted rule is determined by the rule whose boundary contains the predicted start position:

$$\hat{r} = \text{boundary}(\hat{k}^{\text{start}}) \quad (3.3)$$

In the multiple-choice formulation, each candidate rule is paired with the input comment and evaluated independently by the model. The model assigns a relevance score to each comment-rule pair and produces a probability distribution over the candidate rules using a softmax layer:

$$P(r_i | c, R) = \frac{\exp(s_i)}{\sum_{j=1}^n \exp(s_j)} \quad (3.4)$$

where s_i is the relevance score assigned to rule r_i given comment c , and n is the number of candidate rules in R . The rule with the highest probability is selected as the predicted violation:

$$\hat{r} = \arg \max_{r_i \in R} P(r_i | c, R) \quad (3.5)$$

From a modeling perspective, this formulation can be interpreted as a ranking problem over dynamically provided rule sets, implemented as multiple-choice classification with cross-entropy optimization. [26, 27].

These two formulations provide complementary perspectives on rule violation detection. The extractive formulation leverages span prediction over structured rule representations, while the multiple-choice formulation models competition between candidate rules. Comparing these approaches allows us to evaluate how different task representations influence model performance and generalization across communities and previously unseen rule sets.

3.2 Dataset and splits

The experiments in this study are conducted on a dataset of moderation decisions collected from Lemmy, a federated open-source social platform organized into topic-specific communities, each governed by its own moderation rules.¹ Each instance consists of a user comment, a set of community-specific rules, and the rule applied by moderators when removing the comment. Moderator removal decisions serve as ground truth labels, under the assumption that they reflect the rule the moderator judged to have been violated. Table 3.1 presents a representative example from the raw dataset.

Field	Value
ap_id	https://reddthat.com/comment/3579927
content	“Pretty much. It’s a tankie troll. Only way to solve the problem is to refuse to feed the troll.”
community.name	memes
community.rules	{“0”: “Safe”, “1”: “Be civil and nice.”, “2”: “Try not to excessively repost...”}
applied_rule_n	1
applied_rule_text	“Be civil and nice.”
removed	true

Table 3.1. A raw dataset instance in its original JSON structure. Fields such as `ap_id`, `mod_person_id`, and `modlog_id` are metadata not used during model training. The fields consumed by the model are `content`, `community.name`, `community.rules`, and `applied_rule_text`.

The rule set is provided as part of the input for each example rather than being fixed across the dataset. As a result, the model must select the violated rule from a dynamically defined candidate set whose size and content vary between communities.

Each rule set additionally includes a special rule labeled *Safe*, which represents comments that do not violate any moderation policy. This allows the prediction task to be formulated as selecting the most appropriate rule regardless of whether a violation is present.

¹Lemmy (<https://join-lemmy.org>) is a federated, open-source link aggregator and discussion platform, similar in structure to Reddit, where communities are independently hosted and moderated.

The dataset reflects realistic moderation scenarios in which communities define their own rule systems using natural language descriptions. For example, one community may include rules such as *Be civil* or *No misinformation*, while another may define restrictions related to topic relevance or reposting behavior. Consequently, rule identifiers are not shared globally across communities and must be interpreted in context.

To evaluate model performance under different generalization conditions, three evaluation splits are used in this work, as summarized in Table 3.2:

- The stratified test split preserves the overall distribution of communities and rule types observed during training and represents the standard in-distribution evaluation setting.
- The unseen rules split excludes rules appearing in the test set from the training data. This setting evaluates whether the model can generalize to previously unseen rule descriptions rather than relying on memorized rule patterns.
- The unseen communities split excludes entire communities from the training data and evaluates whether the model can transfer rule violation detection capabilities to new moderation environments with different rule formulations.

Split	Training set contains	Test set contains	Example
Stratified	Communities: <i>memes</i> , <i>linux</i> , <i>technology</i> Rules: <i>Be civil and nice.</i> , <i>No misinformation</i>	Communities: <i>memes</i> , <i>linux</i> , <i>technology</i> Rules: <i>Be civil and nice.</i> , <i>No misinformation</i>	Comment from <i>memes</i> violating <i>Be civil and nice.</i> - a rule seen during training
Unseen rules	Communities: <i>memes</i> , <i>linux</i> , <i>technology</i> Rules: <i>Be civil and nice.</i>	Communities: <i>memes</i> , <i>linux</i> , <i>technology</i> Rules: <i>No misinformation</i>	Comment from <i>linux</i> violating <i>No misinformation</i> - a rule never seen during training
Unseen communities	Communities: <i>memes</i> , <i>linux</i> Rules: all their rules	Communities: <i>technology</i> Rules: all its rules	Comment from <i>technology</i> - a community entirely excluded from training

Table 3.2. Illustration of the three evaluation splits using examples from the dataset. The stratified split evaluates in-distribution performance, the unseen rules split evaluates generalization to new rule descriptions, and the unseen communities split evaluates transfer to entirely new moderation environments.

Together, these evaluation settings provide a comprehensive assessment of model performance across in-distribution prediction, rule-level generalization, and cross-community transfer.

3.3 Input representation

In order to apply transformer-based language models to the rule violation detection task, each training instance is converted into a structured textual sequence that combines the user comment with the corresponding community rules. To illustrate how a raw instance is transformed into model input, Table 3.3 shows the formatted sequences derived from the example in Table 3.1.

Since rule sets differ across communities in both size and content, the input representation must allow the model to process multiple candidate rules together with the comment within a single sequence. To achieve this, special boundary tokens are introduced to separate rules and comments and to preserve their structure during encoding.

In the extractive question answering formulation, each example is formatted as a single structured sequence:

$$\begin{aligned}
 &[BOK] \text{ community_name } [EOK] \\
 &\quad [BOR] \text{ rule}_1 [EOR] \\
 &\quad [BOR] \text{ rule}_2 [EOR] \\
 &\quad \quad \quad \vdots \\
 &\quad [BOR] \text{ rule}_n [EOR] \\
 &[BOC] \text{ comment } [EOC]
 \end{aligned}$$

where $[BOR]$ and $[EOR]$ denote the beginning and end of a rule, $[BOC]$ and $[EOC]$ denote the beginning and end of the comment, and $[BOK]$ and $[EOK]$ denote the beginning and end of the community name.

In the multiple-choice formulation, candidate rules are instead paired individually with the input comment and encoded as separate input sequences. Formally, the input for each candidate rule takes the following form:

$$[CLS] \text{ comment } [SEP] \text{ community_name } [SEP] \text{ rule}_i [SEP]$$

where the classification token $[CLS]$ is used to obtain a joint representation of the comment and the candidate rule. This representation is then used to compute a relevance score for rule selection.

3.4 Model architectures

This section describes the concrete architectural choices made for each of the two formulations introduced in Section 3.1. Both models share the same pretrained *bert-base-uncased* backbone[18] but differ in how the input is structured and how the final prediction is produced.

Formulation	Formatted Input
Multiple-choice	[CLS] Pretty much. It's a tankie troll... [SEP] memes [SEP] Be civil and nice. [SEP] [CLS] Pretty much. It's a tankie troll... [SEP] memes [SEP] Try not to excessively repost... [SEP] [CLS] Pretty much. It's a tankie troll... [SEP] memes [SEP] Safe [SEP]
Extractive QA	[BOK] memes [EOK] [BOR] Be civil and nice. [EOR] [BOR] Try not to excessively repost... [EOR] [BOR] Safe [EOR] [BOC] Pretty much. It's a tankie troll... [EOC]

Table 3.3. The same instance formatted as model input for the multiple-choice formulation (one sequence per candidate rule) and the extractive QA formulation (single structured sequence). The correct answer is *Be civil and nice.* in both cases.

3.4.1 Extractive question answering formulation

The extractive model is built on *bert-base-uncased*, a 12-layer bidirectional transformer encoder extended with custom special tokens introduced in Section 3.3. As illustrated in Figure 3.1, the architecture processes the structured input sequence in three stages.

First, the community rules and user comment are combined into a single structured sequence using the boundary tokens defined in Section 3.3, and passed through the BERT encoder to produce contextual token representations h_k for each position k .

Second, two independent linear projection layers are applied on top of the encoder output to compute start and end logits over all token positions, as defined in Equations 3.1 and 3.2. These logits represent the model's confidence that a given token marks the beginning or end of the violated rule span.

Third, the predicted token positions are passed through a rule boundary mapping step that converts them into a rule index. Specifically, the model identifies which rule's token span contains the predicted start position, and returns that rule as the final prediction, as defined in Equation 3.3.

During training, token-level boundaries for each rule are provided as supervision signals, allowing the model to learn associations between comment semantics and rule descriptions through span prediction rather than fixed-label classification. This formulation naturally supports variable-length rule sets and enables generalization to previously unseen rules by relying on their textual descriptions rather than memorized rule identifiers [28].

The complete implementation of this mapping strategy is described in Section [Span-level evaluation](#).

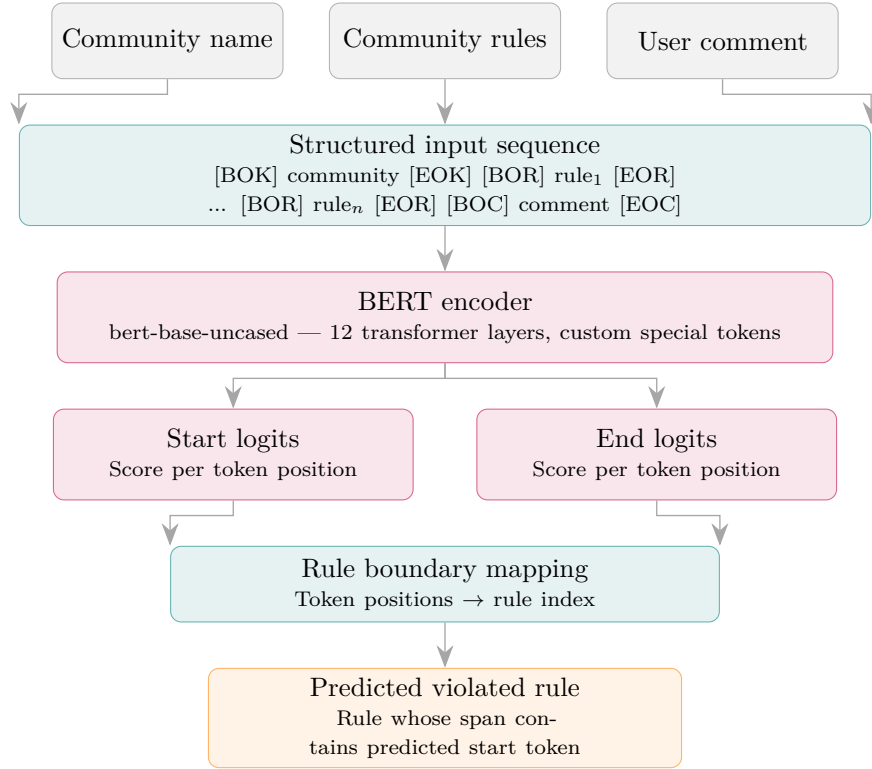


Figure 3.1. Extractive QA model architecture. The structured input sequence combines the community name, candidate rules, and user comment using custom boundary tokens.

3.4.2 Multiple-choice formulation

In the multiple-choice formulation, rule violation detection is modeled as a candidate selection task in which each rule is evaluated independently with respect to the input comment. Unlike the extractive formulation, which predicts span boundaries within a single structured sequence of all rules, the multiple-choice formulation evaluates each candidate rule separately, allowing direct comparison between alternative rule descriptions.

For each training instance, the model processes pairs consisting of the user comment, the community name, and a candidate rule description. As illustrated in Figure 3.2, each pair is encoded jointly as a single input sequence using the *bert-base-uncased* encoder. The contextual representation associated with the *[CLS]* token is passed through a linear classifier, producing a relevance score for each candidate rule.

The scores assigned to all candidate rules within the same example are normalized using the softmax layer defined in Equation 3.4, producing a probability distribution over the rule set. The rule with the highest probability is selected as the predicted violation, as defined in Equation 3.5. During training, the model parameters are optimized by minimizing the cross-entropy loss:

$$\mathcal{L} = -\log P(r^* \mid c, R) \quad (3.6)$$

where r^* denotes the ground-truth violated rule. Similar candidate selection formulations are widely used in transformer-based answer selection tasks that require

comparing multiple textual alternatives within a shared semantic space [26].

This formulation makes it possible to interpret rule violation detection as a ranking problem over dynamically defined rule sets. Because each comment–rule pair is encoded independently, the model can directly compare alternative rule descriptions and generalize to variable-length rule sets without requiring modifications to the input structure.

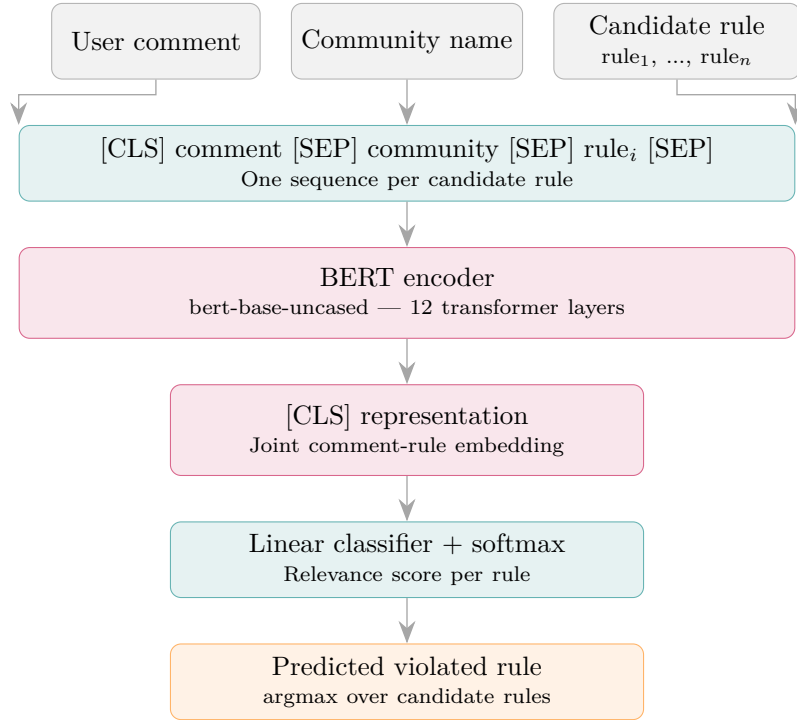


Figure 3.2. Multiple-choice model architecture.

Chapter 4

Training setup

This chapter describes the experimental configuration used to train and evaluate the models. Section 4.1 covers the implementation details: model initialization, tokenization, optimizer settings, data loading, and data augmentation strategies. Section 4.2 defines the evaluation metrics used throughout the experiments. Section 4.3 describes the four baseline models used as reference points. Section 4.4 presents the hyperparameter selection experiments, and Section 4.5 presents the augmentation selection experiments. Finally, Section 4.6 describes how the extractive model's training configuration was derived from the multiple-choice experiments.

4.1 Training configuration

4.1.1 Model initialization and tokenization

Both extractive and multiple-choice formulations were implemented using pretrained transformer encoders initialized from the *bert-base-uncased* checkpoint. The special tokens were registered using the HuggingFace tokenizer API as follows:

```
# Extractive QA tokenizer
tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased")
tokenizer.add_special_tokens({
    "additional_special_tokens": [
        "[BOR]", "[EOR]", "[BOC]", "[EOC]",
        "[BOK]", "[EOK]"
    ]
})
model.resize_token_embeddings(len(tokenizer))

# Multiple-choice tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
context = example["content"] + " [SEP] " + example["community"]["name"]
```

The tokenizer for the extractive formulation was extended with special tokens *[BOR]*, *[EOR]*, *[BOC]*, and *[EOC]*, which denote the boundaries of rules and comments respectively.

The call to `resize_token_embeddings` is necessary for the extractive formulation to extend the model's embedding matrix to account for the newly added special tokens, ensuring that each receives a dedicated learnable representation during fine-tuning. The multiple-choice formulation does not require this step as no new tokens are introduced.

Extractive formulation tokenization

In the extractive formulation, tokenization is applied to the full dataset at once using the HuggingFace `datasets.Dataset.map()` function, which processes all examples in parallel across 4 processes:

```
tokenized_dataset = dataset_qa.map(
    partial(tokenize, tokenizer=tokenizer,
            truncation="only_second"),
    batched=True,
    remove_columns=dataset_qa.column_names,
    num_proc=4,
)
dataset = tokenized_dataset.with_format("torch")
```

The `truncation="only_second"` argument ensures that only the comment portion is truncated if the input exceeds the maximum sequence length, preserving the full rule descriptions in the input. Rule token boundaries are tracked during tokenization and stored alongside each example to enable span-to-rule mapping during evaluation.

Multiple-choice formulation tokenization

The multiple-choice formulation used the standard `[CLS]` and `[SEP]` tokens provided by the pretrained BERT tokenizer. An additional `[SEP]` token was inserted during preprocessing to separate the comment from the community name before tokenization, resulting in the following input structure:

`[CLS] comment [SEP] community_name [SEP] rulei [SEP]`

where `[CLS]` and the final `[SEP]` are inserted automatically by the tokenizer, while the intermediate `[SEP]` between the comment and community name is introduced during preprocessing.

Unlike the extractive formulation, tokenization in the multiple-choice formulation is performed on-the-fly for each individual example during training, inside the `__getitem__` method of the `RuleQADataset` class. For a community with n rules, this produces n separate input sequences per example, each padded to a fixed maximum length of 128 tokens:

```
def __getitem__(self, idx):
    example = self.data[idx]
    context = (example["content"] + " [SEP] " +
               example["community"]["name"])
```

```

choices = list(example["community"]["rules"].values())
label = choices.index(example["applied_rule_text"])
encodings = self.tokenizer(
    [context] * len(choices),
    choices,
    truncation=True,
    padding="max_length",
    max_length=128,
    return_tensors="pt"
)
return {
    "input_ids": encodings["input_ids"],
    "attention_mask": encodings["attention_mask"],
    "label": label,
    "num_choices": len(choices)
}

```

The label is the position of the violated rule within the candidate rule list. The same comment is repeated n times – once for each candidate rule – so that the tokenizer produces n independent input sequences, each pairing the comment with a different rule.

4.1.2 Optimizer and training stability

Model parameters were optimized using the AdamW optimizer in both formulations. Learning rate and weight decay coefficients were varied across experiments to analyze their influence on model convergence and generalization performance. The final configurations were selected based on validation performance.

In the multiple-choice formulation, training was handled by the HuggingFace *Trainer* for 5 to 10 epochs. The batch size was set to 2 per device due to memory constraints caused by structured rule representations and multiple candidate inputs per example. Mixed precision training was enabled via *fp16*, reducing memory consumption and accelerating training by performing forward and backward passes in 16-bit floating point. Gradient clipping was applied automatically by the Trainer with a maximum gradient norm of 1.0.

```

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    num_train_epochs=10,
    weight_decay=0.01,
    fp16=True,
    max_grad_norm=1.0,
    metric_for_best_model="eval_f1",
    greater_is_better=True,
)

```


In the extractive formulation, training was implemented as a manual loop for 10 epochs with a batch size of 16. The larger batch size compared to the multiple-choice formulation was possible because each training instance is encoded as a single structured sequence rather than multiple candidate-rule pairs. The loss was computed at each step as the sum of cross-entropy losses over the start and end positions independently:

$$\mathcal{L} = \mathcal{L}_{\text{start}} + \mathcal{L}_{\text{end}} \quad (4.1)$$

The loss was scaled by the number of gradient accumulation steps before back-propagation, and both `optimizer.step()` and GrokFast¹ were applied only at the corresponding accumulation boundary. Gradient clipping was applied manually with a maximum gradient norm of 1.0:

```
grads = None
for epoch in range(n_epochs):
    for batch in train_loader:
        optimizer.zero_grad()
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]
        start_positions = batch["start_positions"]
        end_positions = batch["end_positions"]
        outputs = model(
            input_ids,
            attention_mask=attention_mask,
            start_positions=start_positions,
            end_positions=end_positions,
        )
        loss_value = loss(outputs["start_logits"], start_positions) + \
            loss(outputs["end_logits"], end_positions)
        loss_value = loss_value / GRADIENT_ACC_STEPS
        loss_value.backward()
        if steps % GRADIENT_ACC_STEPS == 0:
            optimizer.step()
            grads = gradfilter_ema(model, grads=grads)
        steps += 1
```

At each accumulation boundary, `gradfilter_ema` computes an exponential moving average of the gradients and amplifies the slow-varying components. It has been shown to accelerate the transition from memorization to generalization during fine-tuning.

4.1.3 Data loading and batching

The two formulations differ in how training data are loaded and batched, owing to their different input structures.

¹GrokFast [29] is a gradient filtering technique based on exponential moving average of gradients that accelerates generalization by amplifying low-frequency gradient components.

Multiple-choice formulation

Since different communities define different numbers of candidate rules, samples collected by the *DataLoader* cannot be stacked directly into tensors of uniform shape. A custom *VariableChoiceCollator* is therefore applied after tokenization to pad each example's rule dimension to the maximum number of choices present across all dataset splits:

```
class VariableChoiceCollator:
    def __init__(self, max_num_choices):
        self.max_num_choices = max_num_choices

    def __call__(self, features):
        max_choices = self.max_num_choices
        max_len = features[0]["input_ids"].shape[1]

        def pad_tensor(tensor, target_shape):
            pad_size = target_shape[0] - tensor.shape[0]
            return torch.nn.functional.pad(
                tensor, (0, 0, 0, pad_size), value=0
            )

        input_ids = torch.stack([
            pad_tensor(f["input_ids"], (max_choices, max_len))
            for f in features
        ])
        attention_mask = torch.stack([
            pad_tensor(f["attention_mask"], (max_choices, max_len))
            for f in features
        ])
        labels = torch.tensor([f["label"] for f in features])

        return {
            "input_ids": input_ids,
            "attention_mask": attention_mask,
            "labels": labels
        }
```

Padding is applied along the choice dimension using zeros, which are masked out by the attention mechanism and therefore do not influence the model's predictions. The maximum number of choices is computed globally across all dataset splits before training begins, ensuring consistent tensor shapes throughout:

```
max_num_choices = get_max_num_choices([train_data,
                                         eval_data,
                                         test_data])

def get_max_num_choices(datasets):
    return max(len(example["community"]["rules"])
               for dataset in datasets
               for example in dataset)
```

The collator is connected to the training pipeline through the HuggingFace *Trainer*, which internally creates a *DataLoader*. At each training step, the *DataLoader* calls `__getitem__` for each sample index in the batch, collects the tokenized examples into a list, and passes them to *VariableChoiceCollator* to produce the final padded batch tensors before forwarding them to the model:

```
data_collator = VariableChoiceCollator(
    max_num_choices=max_num_choices)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    processing_class=tokenizer,
    compute_metrics=compute_metrics,
    data_collator=data_collator
)
```

Extractive formulation

In the extractive formulation, since all examples are encoded as a single structured sequence of the same shape, no custom collator was required. A standard PyTorch *DataLoader* with a batch size of 16 was used directly:

```
train_loader = DataLoader(dataset, batch_size=16, shuffle=True)
```

Rule token boundaries are tracked during tokenization and stored alongside each example. At evaluation time, these boundaries are used to map the model's predicted span back to the corresponding rule index.

4.1.4 Data augmentation

Three augmentation strategies were applied to the training data to both the multiple-choice and extractive formulations. Each was tested separately and in combination to see which ones actually help and which ones do not.

Rule shuffling

The order of candidate rules within each example is randomly permuted at each training step. Without this strategy, the model may exploit positional regularities in the training data: early rules tend to be more general in scope and more frequently violated, making position a spurious but reliable signal for prediction. By randomizing rule order, the model is forced to rely on semantic matching between the comment and rule descriptions rather than on positional shortcuts.

Number permutation

Comments sometimes contain numbers, post counts, timestamps, vote tallies. This strategy perturbs them slightly so the model does not memorize specific numeric patterns from training examples. Short numbers (2 digits or fewer) get a random delta of -1 , 0 , or $+1$. Longer ones get their digits shuffled, with a check to avoid leading zeros.

Rule exclusion

Some communities have 8-10 rules. A model trained on such examples might learn to narrow down the answer by process of elimination rather than by actually matching the comment to the violated rule. Rule exclusion addresses this by randomly dropping some of the non-violated rules from the candidate list, but always keeps the correct one and at least one distractor (controlled by `min_choices=2`).

Augmentation pipeline

All three strategies are applied through a unified augmentation pipeline controlled by boolean flags. Each augmented example is checked against a fingerprint of its comment content and rule order to prevent duplicates. For instance, this can occur when shuffling produces the same rule order as the original. In the final experiments, `n_replicas_per_strategy` was set to 1, so each training example produces at most one augmented copy per active strategy.

The full implementation is provided in Appendix [.1](#).

4.1.5 Computational setup

All experiments were conducted on a single NVIDIA A100 GPU with 40 GB of memory, accessed via Google Colab Pro. The experimental workflow followed a two-stage approach.

In the first stage, hyperparameter and data augmentation selection was conducted exclusively on the multiple-choice model. Running the full search on both models simultaneously would have been computationally prohibitive, since each epoch of the multiple-choice model took approximately 20–25 minutes with a batch size of 2. This overhead stems directly from encoding each candidate rule as a separate input sequence: a community with 8 rules produces 8 forward passes per example instead of one. The selection stage therefore used 5 epochs per configuration to keep the search tractable.

In the second stage, the two best-performing configurations identified during selection were applied to both models for the final experiments. Both were trained for 10 epochs to ensure comparability. The extractive model used a batch size of 16, with each epoch taking approximately 17 minutes. This is faster per epoch than the multiple-choice model, despite processing more examples per batch, because each training instance is a single sequence rather than a set of n candidate-rule pairs.

4.2 Evaluation metrics

4.2.1 Classification metrics

Model performance was evaluated using two metrics: accuracy and macro-averaged F1 score. Accuracy measures the proportion of correctly predicted rule labels across all test examples. However, accuracy alone is insufficient here, since rule sets vary in size between communities (from 2 to 11 rules), and some rules appear far more frequently than others in the dataset. A model that always predicts the most common rule would achieve high accuracy without learning anything meaningful about rule semantics.

Macro-averaged F1 addresses class imbalance by computing the F1 score independently for each rule class.² The results are then averaged to provide equal weight to frequent and rare rules alike. This metric has one limitation in the present setting. Since rule indices are community-specific — the same index refers to different rules in different communities — macro-F1 ends up averaging across heterogeneous categories rather than truly comparable rule classes. Both metrics were computed using the following function:

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    preds = predictions.argmax(-1)
    return {
        "accuracy": accuracy_score(labels, preds),
        "f1": f1_score(labels, preds, average="macro")
    }
```

Macro-F1 was selected as the primary evaluation metric throughout all experiments. Accuracy is reported alongside it for reference, but all model comparisons and conclusions in this thesis are based on macro-F1.

Span-level evaluation

The extractive formulation requires a separate evaluation procedure. Unlike the multiple-choice model, which directly outputs a probability distribution over candidate rules, the extractive model outputs start and end logits over all token positions in the input sequence. The predicted rule is then recovered by checking which rule’s token span contains the predicted position — a mapping step that introduces an additional source of error beyond the model’s raw prediction quality.³

Rule boundaries are tracked during tokenization and stored alongside each example. At evaluation time, the predicted start and end positions are compared against the stored boundaries for each rule:

²Macro-F1 is particularly sensitive to performance on minority classes. In a dataset where one rule accounts for 40% of violations, a model ignoring all other rules would still achieve 0.40 accuracy but would receive a much lower macro-F1, exposing the imbalance.

³A model can predict the correct general region of the input but still miss the exact rule boundary, for instance by landing on a boundary token such as *[BOR]* or *[EOR]* rather than within the rule text itself.

```

for rule, rule_boundaries in batch['rule_token_boundaries'].items():
    for i, s, e, (rs, re) in zip(
        range(len(rule_boundaries)),
        outputs['start_logits'].argmax(1),
        outputs['end_logits'].argmax(1),
        rule_boundaries
    ):
        s, e, rs, re = s.item(), e.item(), rs.item(), re.item()
        if rs <= s < re:
            predicted_rules_by_start[i] = int(rule)
        if rs < e <= re:
            predicted_rules_by_end[i] = int(rule)

```

Predictions are derived from start and end logits independently, producing two separate sets of rule predictions per example. A prediction is considered correct if the predicted token position falls strictly within the token boundaries of the correct rule segment – boundary tokens themselves are excluded. This allows separate analysis of whether the model is better at identifying where a violated rule begins versus where it ends, which can reveal asymmetries in how the model localizes rule spans.

To evaluate generalization, all models – both extractive and multiple-choice – were tested on three dataset splits described in Section 3.2: the stratified split for in-distribution performance, the unseen rules split for generalization to new rule descriptions, and the unseen communities split for cross-community transfer. The same three splits are used for both formulations, ensuring that all comparisons between models are made under identical evaluation conditions.

4.3 Baseline models

To evaluate whether transformer-based models capture meaningful relationships between comments and community rules, four baseline strategies were implemented as reference points, ranging from simple frequency-based predictors to traditional machine learning classifiers. None of the baselines use rule descriptions as input – they operate on comment text only, which makes them a direct test of whether semantic matching between comments and rules is actually necessary.

The **most frequent** baseline always predicts the rule observed most frequently in the training data, implemented via scikit-learn’s *DummyClassifier* with *strategy="most_frequent"*, ignoring both the comment and the rule descriptions entirely. On the stratified split, this achieves 0.460 accuracy – which sounds reasonable until one notices the macro-F1 of just 0.057, revealing that it essentially never predicts minority rules correctly.

The **random** baseline selects a rule using stratified sampling – meaning rules are sampled proportionally to their frequency in the training data rather than uniformly at random.⁴ It is implemented via *DummyClassifier* with *strategy="stratified"* and represents the floor: if a model cannot beat this, something is fundamentally wrong with the setup.

⁴Stratified sampling here means the random baseline respects the class distribution of the training set. A rule that appears in 30% of training examples will be predicted 30% of the time on average.

The **Naive Bayes** baseline uses a Complement Naive Bayes classifier⁵ with TF-IDF features extracted from comment text, treating rule violation detection as standard text classification with *max_features=1000*.

The **Linear SVM** baseline uses a Linear Support Vector Machine with hashed n-gram features (*n_features=1000*) extracted from comment text. It includes an edge case handler for training splits where only a single rule class is present — in such cases, the classifier returns that class directly without fitting, avoiding a scikit-learn error. It is the strongest of the four baselines and serves as the most meaningful point of comparison for the transformer-based models.

Both the Naive Bayes and Linear SVM baselines were implemented using the scikit-learn library as follows:

```
# Naive Bayes
pipeline_nb = Pipeline([
    ("vect", TfidfVectorizer(max_features=1000)),
    ("clf", ComplementNB()),
])

# Linear SVM
pipeline_svm = Pipeline([
    ("vect", HashingVectorizer(n_features=1000)),
    ("clf", LinearSVC()),
])
```

Table 4.1 presents the performance of all four baseline strategies across the three evaluation splits.

Baseline	Evaluation split	Accuracy	Macro-F1
Random	Stratified	0.295	0.083
Random	Unseen rules	0.167	0.076
Random	Unseen communities	0.200	0.055
Most Frequent	Stratified	0.460	0.057
Most Frequent	Unseen rules	0.350	0.086
Most Frequent	Unseen communities	0.085	0.031
Naive Bayes	Stratified	0.519	0.159
Naive Bayes	Unseen rules	0.250	0.091
Naive Bayes	Unseen communities	0.247	0.108
Linear SVM	Stratified	0.529	0.173
Linear SVM	Unseen rules	0.350	0.196
Linear SVM	Unseen communities	0.262	0.116

Table 4.1. Baseline model performance across evaluation splits.

The results expose a clear pattern. On the stratified split, the Linear SVM reaches 0.173 macro-F1 – the strongest baseline result, but still 0.518 points below the multiple-choice model. On the unseen-communities split, all baselines collapse: the most frequent predictor drops to 0.031 macro-F1, confirming that label frequency patterns learned during training do not transfer across communities.

⁵Complement Naive Bayes is a variant of the standard Naive Bayes classifier particularly suited for imbalanced datasets, as it estimates class probabilities using the complement of each class rather than the class itself.

The more interesting finding is on the unseen-rules split. Here the Linear SVM achieves 0.196 macro-F1 – marginally higher than both transformer-based models (0.193 for multiple-choice, 0.144 for extractive). This is not a coincidence. The SVM operates purely on comment text, without ever reading the rule descriptions. When the rules in the test set are entirely new, the transformer models lose their main advantage – the ability to match comments against rule text – and fall back to pattern matching over comment content, where the SVM turns out to be competitive. This result foreshadows the discussion in Chapter [Experiments and Evaluation](#) on the limits of fine-tuning for generalization to unseen rules.

The full baseline implementation is provided in [Appendix .3](#).

4.4 Hyperparameter selection

Hyperparameter selection was conducted on the multiple-choice model during the first experimental stage, as described in [Section 4.1.5](#). Three learning rate values were evaluated — 2×10^{-5} , 1×10^{-5} , and 5×10^{-6} — combined with weight decay values of 0.00, 0.01, 0.02, 0.03, and 0.10, giving 8 configurations in total. These learning rate values are consistent with the fine-tuning range reported in the original BERT paper, where learning rates between 2×10^{-5} and 5×10^{-5} were used for downstream tasks [\[18\]](#).

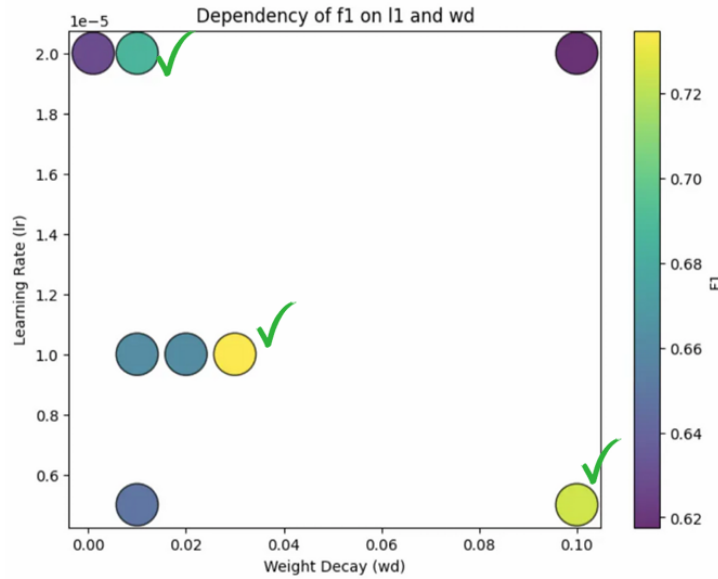


Figure 4.1. Macro-F1 scores on the stratified split for different learning rate and weight decay configurations.

Figure [4.1](#) show macro-F1 on the stratified split for all 8 tested configurations. Three of them stand out with scores above 0.69: 2×10^{-5} with weight decay 0.01 (0.69), 1×10^{-5} with weight decay 0.03 (0.72), and 5×10^{-6} with weight decay 0.1 (0.73). The remaining five configurations score between 0.62 and 0.67.

A consistent pattern emerges: as the learning rate decreases, the optimal weight

decay increases. For $lr = 2 \times 10^{-5}$, the best weight decay is 0.01 – higher values hurt performance (0.63 at $wd = 0.1$). For $lr = 5 \times 10^{-6}$, the best weight decay jumps to 0.1, producing the highest score on the stratified split overall (0.73). This is expected, since a smaller learning rate means slower parameter updates. It requires stronger regularization to prevent the model from drifting during the later epochs of fine-tuning.

The three configurations with the highest stratified macro-F1 were then evaluated on the unseen-rules and unseen-communities splits to assess their generalization performance, as shown in Figure 4.2.

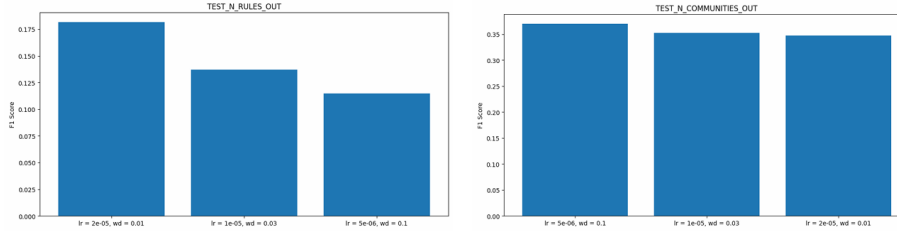


Figure 4.2. Comparison of hyperparameter configurations on unseen-rules and unseen-communities splits.

On the unseen-rules split, the ranking reverses completely compared to the stratified split. The configuration that scored highest on the stratified split – 5×10^{-6} with weight decay 0.1 (0.73) – drops to the lowest macro-F1 on unseen rules (0.117). The configuration 2×10^{-5} with weight decay 0.01, which scored only 0.69 on the stratified split, achieves the best generalization to new rule descriptions (0.180). This reversal suggests that the 5×10^{-6} configuration overfits to rule patterns seen during training: it learns them well enough to score highest in-distribution, but fails to transfer that knowledge to unseen rule formulations.

On the unseen-communities split, all three configurations achieve substantially higher scores than on the unseen-rules split – 0.347, 0.351, and 0.362 respectively – suggesting that transferring to new communities is a less difficult challenge than generalizing to entirely new rule descriptions. More importantly, the gap between the best and worst configuration is only 0.015 points, compared to 0.063 points on the unseen-rules split. Tuning hyperparameters simply does not move the needle on unseen communities – the model performs similarly regardless of which configuration is used.

Based on these results, two configurations were selected for the final experiments: 2×10^{-5} with weight decay 0.01 and 1×10^{-5} with weight decay 0.03. Both show strong and consistent performance across all three splits. The third configuration, 5×10^{-6} with weight decay 0.1, was excluded despite achieving the highest stratified score (0.73). Its collapse on the unseen-rules split (0.117) makes it unsuitable for the generalization-focused evaluation that is central to this thesis.

4.5 Data augmentation selection

To evaluate the influence of data augmentation on model performance, each augmentation strategy was tested separately using the two hyperparameter configurations

selected in the previous section: 2×10^{-5} with weight decay 0.01 and 1×10^{-5} with weight decay 0.03. Testing each strategy in isolation makes it possible to attribute performance changes to a specific augmentation rather than to their interaction.

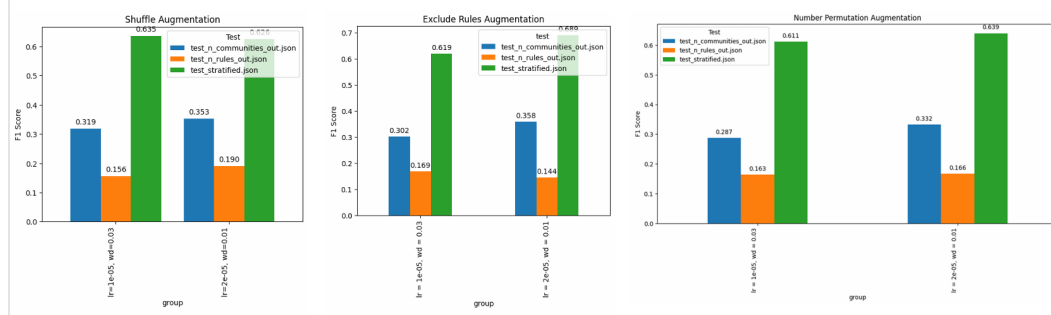


Figure 4.3. Effect of augmentation on model performance across evaluation splits.

Figure 4.3 shows the results. Rule shuffling produces the most consistent gains across both configurations. On the unseen-rules split, macro-F1 reaches 0.190 for $lr = 2 \times 10^{-5}$ and 0.156 for $lr = 1 \times 10^{-5}$ – both improvements over the no-augmentation baseline. On the unseen-communities split, the same configuration achieves 0.353, the highest value among all individual strategies tested. The improvement holds across both configurations, which gives confidence that rule shuffling addresses a real weakness in the model. Without shuffling, the model can exploit positional regularities in the training data, learning to favour rules that appear at a fixed position rather than reasoning over their textual content. Shuffling removes that shortcut.

Rule exclusion shows a more mixed picture. For $lr = 2 \times 10^{-5}$, it achieves the highest stratified score of all strategies (0.689), but drops to 0.144 on the unseen-rules split – lower than both rule shuffling (0.190) and number permutation (0.166) under the same configuration. For $lr = 1 \times 10^{-5}$, the pattern partially reverses: unseen-rules performance improves to 0.169, but the stratified score drops to 0.619. This inconsistency across configurations makes rule exclusion unreliable as a standalone strategy.

Number permutation produces the weaker results overall. On the unseen-communities split, it scores 0.332 and 0.287, the lowest values among all three strategies. Perturbing numerical expressions in comments appears to add noise without meaningfully improving the model’s ability to match comments against unseen rule descriptions.

For $lr = 2 \times 10^{-5}$ with weight decay 0.01, combining all strategies drops the stratified macro-F1 to 0.599 – a sharp decline from 0.626 achieved by rule shuffling alone under the same configuration. The unseen-rules score improves slightly to 0.175 compared to rule exclusion (0.144) and number permutation (0.166), but remains below rule shuffling alone (0.190). In other words, combining strategies hurts in-distribution performance without meaningfully improving out-of-distribution generalization.

For $lr = 1 \times 10^{-5}$ with weight decay 0.03, the combined strategy achieves 0.668 on the stratified split – higher than any individual strategy under this configuration. However, the unseen-rules score drops to 0.153, below rule shuffling (0.156), rule exclusion (0.169), and number permutation (0.163). The unseen-communities score of 0.342 also falls below rule shuffling (0.319) and rule exclusion (0.302). The additional diversity introduced by rule exclusion and number permutation appears to add noise that interferes with the positional bias correction provided by rule shuffling, rather

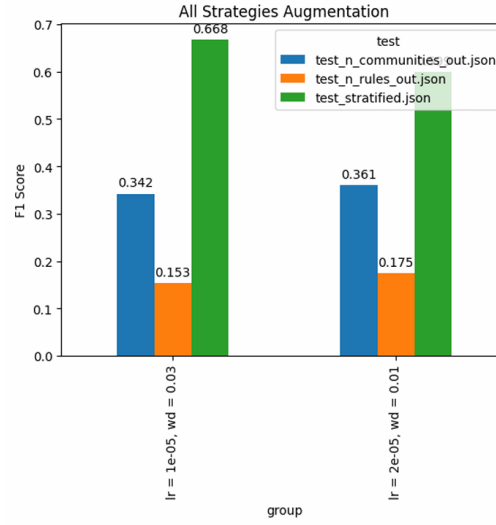


Figure 4.4. Effect of combined augmentation on model performance across evaluation splits.

than compounding with it.

Based on the individual and combined augmentation results, rule-order shuffling was retained as the sole augmentation strategy for the final experiments. It is the only strategy that improves performance consistently across both hyperparameter configurations and both out-of-distribution splits, without introducing the instability observed with rule exclusion, number permutation, or their combination.

4.6 Extractive question answering training configuration

The extractive model was not subjected to an independent hyperparameter search. Running a full grid search on both models would have doubled the computational cost without adding meaningful insight. The goal is to compare two modeling formulations under comparable conditions, not to find the globally optimal configuration for each independently.

Instead, the two configurations that demonstrated the strongest and most consistent generalization during the multiple-choice experiments were transferred directly to the extractive model: $lr = 2 \times 10^{-5}$ with weight decay 0.01, and $lr = 1 \times 10^{-5}$ with weight decay 0.03. The third configuration, $lr = 5 \times 10^{-6}$ with weight decay 0.1, was excluded. It scored highest on the stratified split (0.73) but collapsed on the unseen-rules split (0.117), making it unsuitable for a generalization-focused evaluation.

Rule-order shuffling was adopted as the sole augmentation strategy for the extractive model, for the same reason. It was the only strategy that improved performance consistently across both hyperparameter configurations and both out-of-distribution splits in the multiple-choice experiments. Applying the same augmentation to both models ensures that any performance difference between them reflects a difference in modeling approach rather than a difference in training data.

Chapter 5

Experiments and Evaluation

5.1 Multiple-choice model results

The multiple-choice model was evaluated using the two hyperparameter configurations selected during preliminary experiments: $\text{lr} = 2 \times 10^{-5}$ with weight decay 0.01, and $\text{lr} = 1 \times 10^{-5}$ with weight decay 0.03. Rule-order shuffling was applied as the sole data augmentation strategy. Both configurations were trained for 10 epochs – twice as long as during the selection stage – and evaluated on all three test splits after training completion.

Table 5.1 summarizes the results.

Configuration	Evaluation split	Accuracy	Macro-F1
$\text{lr}=2 \times 10^{-5}$, $\text{wd}=0.01$	Stratified	0.724	0.679
$\text{lr}=2 \times 10^{-5}$, $\text{wd}=0.01$	Unseen rules	0.250	0.193
$\text{lr}=2 \times 10^{-5}$, $\text{wd}=0.01$	Unseen communities	0.586	0.328
$\text{lr}=1 \times 10^{-5}$, $\text{wd}=0.03$	Stratified	0.727	0.691
$\text{lr}=1 \times 10^{-5}$, $\text{wd}=0.03$	Unseen rules	0.183	0.144
$\text{lr}=1 \times 10^{-5}$, $\text{wd}=0.03$	Unseen communities	0.504	0.310

Table 5.1. Multiple-choice model performance after 10 epochs with rule shuffling augmentation.

On the stratified split, both configurations perform similarly – macro-F1 of 0.679 and 0.691 respectively, with accuracy above 0.72 in both cases. The difference of 0.012 macro-F1 points is small enough that neither configuration has a clear advantage under in-distribution conditions.

The two configurations diverge sharply on the unseen-rules split. $\text{lr} = 2 \times 10^{-5}$ with weight decay 0.01 achieves a macro-F1 of 0.193, while $\text{lr} = 1 \times 10^{-5}$ with weight decay 0.03 drops to 0.144 – a gap of 0.049 points. This is consistent with the pattern observed during hyperparameter selection, where the 2×10^{-5} configuration showed stronger transfer to new rule descriptions. The accuracy gap is even more pronounced: 0.250 versus 0.183.

On the unseen-communities split, the same configuration maintains its advantage – 0.328 versus 0.310 macro-F1. Both scores are substantially higher than on the unseen-rules split, confirming the finding from the hyperparameter selection stage

that cross-community transfer is a less difficult challenge than generalizing to entirely new rule descriptions.

Overall, $\text{lr} = 2 \times 10^{-5}$ with weight decay 0.01 is the stronger configuration for generalization purposes. It performs comparably on the stratified split and outperforms the alternative on both out-of-distribution splits.

5.2 Extractive question answering model results

The extractive model was trained under identical conditions to the multiple-choice model: the same two hyperparameter configurations, the same rule-order shuffling augmentation, and the same 10 epochs. One procedural difference: unlike the multiple-choice model, which was evaluated only after training completion, the extractive model was evaluated after each epoch on all three test splits. Results are reported at the epoch achieving the highest macro-F1 per split, which means the reported numbers come from different epochs for different splits – an important detail when interpreting the results.

Table 5.2 presents the best performance achieved by the extractive model across the three evaluation splits.

Configuration	Evaluation split	Best epoch	Accuracy	Macro-F1
$\text{lr}=2 \times 10^{-5}$, $\text{wd}=0.01$	Stratified	8	0.719	0.579
$\text{lr}=2 \times 10^{-5}$, $\text{wd}=0.01$	Unseen rules	2	0.083	0.144
$\text{lr}=2 \times 10^{-5}$, $\text{wd}=0.01$	Unseen communities	1	0.701	0.324
$\text{lr}=1 \times 10^{-5}$, $\text{wd}=0.03$	Stratified	2	0.724	0.588
$\text{lr}=1 \times 10^{-5}$, $\text{wd}=0.03$	Unseen rules	4	0.067	0.116
$\text{lr}=1 \times 10^{-5}$, $\text{wd}=0.03$	Unseen communities	3	0.582	0.317

Table 5.2. Extractive QA model best performance across evaluation splits, selected by highest macro-F1 per split.

On the stratified split, both configurations perform similarly – macro-F1 of 0.579 and 0.588 respectively, with accuracy above 0.71 in both cases. The gap between them is 0.009 points, which is negligible.

The unseen-rules split is where the extractive model struggles most visibly. Both configurations drop below 0.15 macro-F1 – 0.144 for $\text{lr} = 2 \times 10^{-5}$ and 0.116 for $\text{lr} = 1 \times 10^{-5}$. More strikingly, the accuracy for $\text{lr} = 2 \times 10^{-5}$ is just 0.083 – barely above random for a task with multiple candidate rules. This suggests the model is largely unable to transfer span prediction skills to rule descriptions it has never seen during training.

On the unseen-communities split, both configurations recover substantially – 0.324 and 0.317 macro-F1 respectively, with accuracy above 0.58. As with the multiple-choice model, cross-community transfer proves less challenging than cross-rule generalization.

The most striking finding in Table 5.2 is the divergence in best epochs across splits. On the stratified split, the $\text{lr} = 2 \times 10^{-5}$ configuration peaks at epoch 8 – near the end of training. On the unseen-communities split, the same configuration peaks at epoch 1, and on the unseen-rules split at epoch 2. For $\text{lr} = 1 \times 10^{-5}$, the stratified split peaks at epoch 2, while out-of-distribution splits peak at epochs 3 and 4. The

pattern is clear: continued training improves in-distribution performance but hurts generalization. The model learns the training distribution increasingly well, at the cost of transferability to unseen rules and communities. This suggests that early stopping based on out-of-distribution validation performance – rather than stratified performance – could meaningfully improve the extractive model’s generalization without any architectural changes.

5.3 Comparison of modeling approaches

Table 5.3 presents a direct comparison between the best configurations of both modeling approaches and the strongest baseline across all three evaluation splits. For each transformer-based model, the configuration achieving the highest macro-F1 on each split is selected for comparison.

Evaluation split	Linear SVM Macro-F1	Multiple-choice Macro-F1	Extractive QA Macro-F1	MC gain over SVM
Stratified	0.173	0.691	0.588	+0.518
Unseen rules	0.196	0.193	0.144	−0.003
Unseen communities	0.116	0.328	0.324	+0.212

Table 5.3. Comparison of Linear SVM baseline, multiple-choice and extractive QA formulations. MC gain is computed as multiple-choice minus Linear SVM macro-F1.

On the stratified split, the multiple-choice model outperforms the Linear SVM by 0.518 macro-F1 points (0.691 vs 0.173) – a gap large enough to confirm that transformer-based semantic matching between comments and rule descriptions provides substantial value over surface-level text classification. On the unseen-communities split, the advantage holds: +0.212 points over the SVM, with the extractive model performing almost identically to multiple-choice (0.324 vs 0.328).

The unseen-rules split tells a different story. The Linear SVM achieves 0.196 macro-F1 – marginally higher than the multiple-choice model (0.193) and well above the extractive model (0.144). A model that never reads rule descriptions at all outperforms models specifically designed to match comments against rules. The reason is that the transformer models learned associations between comment content and specific rule wordings seen during training. When the test set contains entirely new rule descriptions, those associations are useless. The models fall back to comment-level pattern matching, where the SVM turns out to be competitive. The SVM, having never relied on rule descriptions in the first place, is unaffected by the change.

This result points to a concrete limitation of standard fine-tuning for rule violation detection: encoding rule descriptions together with comments does not guarantee generalization to new rules. The model needs to learn the meaning of rule descriptions rather than their specific wording. This capability likely requires zero-shot or few-shot approaches, contrastive learning objectives, or instruction-tuned models trained to follow natural language directives.

Comparing the two transformer-based formulations directly, the multiple-choice model outperforms the extractive model across all three splits: +0.103 on stratified, +0.049 on unseen rules, and +0.004 on unseen communities. The gap is largest in-distribution and narrows as the evaluation becomes harder. On the unseen-communities split, the two models are essentially equivalent (0.328 vs 0.324).

The likely reason for the multiple-choice advantage is structural. In the extractive formulation, the model must identify the correct span within a single long sequence containing all candidate rules simultaneously: the attention signal for any individual rule is diluted by the presence of all others. The multiple-choice formulation encodes each comment-rule pair independently, allowing focused comparison between the comment and a single rule at a time. For a task that fundamentally requires matching a comment against rule descriptions, evaluating each pair in isolation appears to be the more effective approach.

5.4 Statistical significance testing

The macro-F1 differences between the two models reported in Section 5.3 are computed on finite test sets and could in principle arise by chance. To assess whether they reflect genuine differences in model behavior, McNemar’s test was applied to the predictions of both models on each evaluation split.¹

Statistical comparison was performed using $\text{lr} = 2 \times 10^{-5}$ with weight decay 0.01 for both models — the configuration that demonstrated the strongest generalization to unseen rules for both formulations. The test statistic is computed with continuity correction as:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (5.1)$$

where b is the number of examples correctly classified by the multiple-choice model but not by the extractive model, and c is the number of examples correctly classified by the extractive model but not by the multiple-choice model. A significance threshold of $\alpha = 0.05$ was used throughout.

Table 5.4 presents the results.

Evaluation split	MC only	EQA only	Statistic	p-value
Stratified	296	245	4.621	0.032
Unseen rules	15	2	8.471	0.004
Unseen communities	106	208	32.487	<0.001

Table 5.4. McNemar’s test results comparing multiple-choice (MC) and extractive QA (EQA) models. MC only and EQA only refer to the number of examples correctly classified exclusively by each model.

The differences are statistically significant across all three splits, but the nature of the significance differs between them.

On the stratified split, the multiple-choice model correctly classifies 296 examples that the extractive model fails on, against 245 in the opposite direction ($\chi^2 = 4.621$, $p = 0.032$). The difference is real but modest, since both models get many examples right and wrong, and the advantage of multiple-choice is consistent but not dramatic.

¹McNemar’s test is a non-parametric test for comparing two classifiers on the same set of examples. Unlike paired t-tests, it operates on the contingency table of correct and incorrect predictions rather than on raw scores, making it appropriate for classification tasks where the same test examples are evaluated by both models.

On the unseen-rules split, the numbers are small but the signal is clear: 15 examples exclusively correct for multiple-choice versus only 2 for extractive ($\chi^2 = 8.471$, $p = 0.004$). Both models struggle on this split, but the extractive model essentially stops working – 2 exclusively correct predictions out of the entire test set is close to noise.

The unseen-communities split produces the most counterintuitive result. Despite the multiple-choice model achieving a higher macro-F1 (0.328 vs 0.324), the extractive model exclusively correctly classifies 208 examples compared to only 106 for multiple-choice – nearly twice as many ($\chi^2 = 32.487$, $p < 0.001$). This apparent contradiction resolves when one considers what macro-F1 measures: the multiple-choice model achieves higher macro-F1 because it distributes its correct predictions more evenly across rule categories, while the extractive model concentrates its exclusive correct predictions on a larger number of individual examples but from fewer rule categories. The two models are not just performing differently, they are making qualitatively different errors on unseen communities. It suggests their complementary strengths could be exploited through an ensemble approach.

The implementation of McNemar’s test is provided in Appendix [.2](#).

Chapter 6

Conclusion

6.1 Summary

This thesis investigated the application of transformer-based language models to the task of automatic rule violation detection in online communities. The core challenge is that community rules are written in natural language, vary across communities, and require contextual interpretation. For example, a comment that violates one community’s civility rule may be perfectly acceptable in another. Manual moderation at scale is impractical, yet existing automated approaches based on toxicity classification ignore the specific rule descriptions that moderators actually use to make decisions.

The problem was formulated as a rule selection task: given a user-generated comment and a set of community-specific rules, identify which rule has been violated. Two alternative modeling approaches based on the *bert-base-uncased* architecture were implemented and evaluated. In the extractive formulation, all candidate rules and the comment are concatenated into a single structured sequence using custom boundary tokens, and the model predicts the start and end token positions of the violated rule span. In the multiple-choice formulation, each candidate rule is paired independently with the comment and encoded as a separate input sequence, and the model selects the rule with the highest relevance score after softmax normalization.

Both models were trained on a dataset of moderation decisions collected from Lemmy, a federated open-source social platform, using rule-order shuffling as the primary data augmentation strategy and two hyperparameter configurations selected through systematic preliminary experiments. Evaluation was performed on three dataset splits: a stratified split preserving the original distribution of communities and rules, a split containing previously unseen rule descriptions, and a split containing previously unseen communities. Four baseline strategies were evaluated for comparison: a most-frequent predictor, a random baseline, Naive Bayes with TF-IDF features, and a Linear SVM with hashed n-gram features.

The results tell three distinct stories depending on the evaluation split.

On the stratified split, transformer-based models substantially outperform all baselines. The multiple-choice model achieves 0.691 macro-F1 compared to 0.173 for the strongest baseline (Linear SVM) – a gap of 0.518 points. The extractive model reaches 0.588, outperforming the SVM by 0.415 points. Both differences are statisti-

cally significant by McNemar’s test ($p = 0.032$). This confirms that encoding rule descriptions together with comments provides genuine value for in-distribution rule violation detection.

On the unseen-communities split, both transformer-based models maintain a clear advantage over baselines – 0.328 and 0.324 macro-F1 respectively, compared to 0.116 for the Linear SVM. Cross-community transfer is challenging but tractable. A notable finding from the statistical significance analysis complicates the picture. Despite similar macro-F1 scores, the extractive model exclusively correctly classifies 208 examples compared to only 106 for the multiple-choice model ($\chi^2 = 32.487$, $p < 0.001$). The two models make qualitatively different errors on unseen communities, suggesting their complementary strengths could be exploited through an ensemble approach.

The unseen-rules split is where the results become uncomfortable. The Linear SVM achieves 0.196 macro-F1, marginally outperforming the multiple-choice model (0.193) and well above the extractive model (0.144). A model that never reads rule descriptions outperforms models specifically designed to match comments against rules. One possible explanation is that the transformer models learned associations between comment content and specific rule wordings seen during training, so when test rules are entirely new, those associations no longer apply. An alternative interpretation is that the difficulty goes deeper than the model architecture. Moderation decisions reflect individual moderator judgment, community culture, and norms that evolve over time. These are the factors that a rule description alone cannot fully capture. Even a model that perfectly understood the text of a new rule might still struggle to predict how a specific moderator would apply it in practice. Disentangling these two explanations would require further experiments. Regardless of the underlying explanation, result shows that standard fine-tuning is insufficient for zero-shot generalization to new rule descriptions.

The data augmentation analysis produced one clear finding: rule-order shuffling was the only strategy that improved performance consistently across both hyperparameter configurations and both out-of-distribution splits. Rule exclusion and number permutation showed inconsistent effects through helping in some configurations and hurting in others. Combining all three strategies produced no improvement over shuffling alone and in one case sharply reduced performance (stratified macro-F1 dropping from 0.676 to 0.599 for $\text{lr} = 2 \times 10^{-5}$). The primary bottleneck for generalization is semantic understanding of rule content, not exposure to varied rule orderings.

Several limitations of this work should be acknowledged. The dataset was collected from a single platform (Lemmy), and rule formulations, community cultures, and violation patterns may differ substantially on other platforms such as Reddit or Discord. The annotation process relied on moderator decisions as ground truth labels, which may introduce inconsistency bias. For example, the same comment could plausibly violate multiple rules simultaneously, yet only one label is assigned. The experiments were conducted using `bert-base-uncased`, a relatively small model by current standards (110M parameters), and the hyperparameter search was conducted exclusively on the multiple-choice model and transferred to the extractive model without independent validation, which may have resulted in suboptimal configurations for the extractive formulation. All reported results are based on a single train/test split per evaluation condition, without cross-validation or repeated sampling. Performance estimates may therefore be sensitive to the particular split used — confidence in the reported figures would be strengthened by averaging

over multiple splits or applying k-fold cross-validation. This limitation applies equally to the hyperparameter selection, augmentation selection, and final evaluation experiments. Questions of practical deployment — such as inference latency, fairness considerations, and integration into live moderation pipelines — fall outside the scope of this work and are not addressed experimentally.

6.2 Future work

Four directions stand out as the most consequential extensions of this work.

6.2.1 Zero-shot and few-shot generalization to unseen rules

The most pressing open problem is the unseen-rules split result. A Linear SVM operating purely on comment text outperformed both transformer-based models (0.196 vs 0.193 and 0.144 macro-F1), which means that adding rule descriptions to the input actively hurts generalization when those descriptions are new. Standard fine-tuning teaches the model to associate comment patterns with specific rule wordings and not to reason about what a rule means. Future work should explore approaches that target this capability: contrastive learning objectives that train the model to distinguish violated from non-violated rules based on meaning rather than wording, instruction-tuned models that have been trained to follow natural language directives, or zero-shot and few-shot formulations where the model receives only the rule text at test time without any labeled examples for that rule. A dedicated benchmark for out-of-distribution rule generalization would also help the community measure progress on this specific challenge, which current NLP benchmarks do not address.

6.2.2 Ensemble of multiple-choice and extractive models

The McNemar’s test results on the unseen-communities split reveal something that aggregate metrics hide. The extractive model exclusively correctly classifies 208 examples that the multiple-choice model gets wrong, while the multiple-choice model exclusively correctly classifies only 106 that the extractive model misses. The two models are not just different in performance, they are making different errors on different examples. This is exactly the condition under which ensembling tends to work well. A simple majority vote or learned combination of the two models’ predictions could recover a substantial fraction of the 208 examples the multiple-choice model currently misses on unseen communities, without any additional training data or architectural changes.

6.2.3 Stronger pretrained models

All experiments in this thesis used *bert-base-uncased* (110M parameters), which is small by current standards. Models such as RoBERTa and DeBERTa use improved pretraining objectives and longer training that have been shown to improve performance on semantic matching tasks, which is precisely what rule violation detection requires. Generative models such as instruction-tuned variants of LLaMA or Mistral could be particularly relevant for the unseen-rules challenge, since they are trained

to follow natural language instructions and may be better equipped to reason about the meaning of a rule description rather than its specific wording. The performance gap between the Linear SVM and transformer-based models on unseen rules (0.196 vs 0.193) is narrow enough that a stronger model might close it.

6.2.4 Deployment and real-world integration

This thesis excluded practical deployment from its scope. Several challenges arise when moving from an offline evaluation to a live moderation system. The multiple-choice formulation is computationally expensive at inference time. For example, a community with 10 rules requires 10 forward passes per comment. This cost compounds at the scale of millions of comments per day. Community rules evolve over time, which means the model needs a strategy for incorporating new or updated rules without full retraining. Multilingual communities present an additional challenge, since rule descriptions and user comments may appear in different languages, while *bert-base-uncased* handles only English. Any deployed system also requires human-in-the-loop oversight and fairness auditing to ensure that automated moderation does not systematically disadvantage particular user groups or communities.

Nevertheless, the most natural deployment context for a model of this kind might be a lightweight tool that volunteer moderators could fine-tune and deploy independently for their own community. Fine-tuning requires only a standard laptop and a few hours, and inference can run locally without any external infrastructure. Under this model, each community would maintain its own instance trained on its own rules, preserving the heterogeneity that makes community moderation effective in the first place. The primary prerequisite for this deployment scenario is better generalization to unseen communities, so that a base model could be adapted to a new community's rules with minimal labeled data.

Bibliography

- [1] E. Wulczyn, N. Thain, and L. Dixon, “Ex machina: Personal attacks seen at scale,” in *Proceedings of the 26th International World Wide Web Conference (WWW)*, 2017, pp. 1391–1399.
- [2] T. Gillespie, *Custodians of the Internet: Platforms, Content Moderation, and the Hidden Decisions That Shape Social Media*. Yale University Press, 2018.
- [3] C. Fiesler, J. A. Jiang, J. McCann, K. Frye, and J. R. Brubaker, “Reddit rules! characterizing an ecosystem of governance,” in *Proceedings of ICWSM*, 2018.
- [4] G. Weld and T. Althoff, “Rules and rulers: Quantifying the relationship between communities and their moderators on reddit,” in *Proceedings of ICWSM*, 2024.
- [5] H. Li, B. Hecht, and S. Chancellor, “Measuring the monetary value of online volunteer work,” in *Proceedings of the ACM on Human-Computer Interaction*, vol. 6, no. CSCW2. ACM, 2022.
- [6] E. Chandrasekharan, M. Samory, S. Jhaver, H. Reddy, A. Bruckman, C. Lampe, J. Eisenstein, and E. Gilbert, “The internet’s hidden rules: An empirical study of reddit norm violations at micro, meso, and macro scales,” in *Proceedings of CSCW*, 2018.
- [7] S. Jhaver, D. S. Appling, E. Gilbert, and A. Bruckman, “Does transparency in moderation really matter? user behavior after content removal explanations on reddit,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, pp. 1–27, 2019.
- [8] T. Bajpai and E. Chandrasekharan, “In the queue: Understanding how reddit moderators use the modqueue,” *arXiv preprint arXiv:2509.07314*, 2025.
- [9] ———, “Modeling the modqueue: Towards understanding and improving report resolution on reddit,” *arXiv preprint arXiv:2409.16840*, 2024.
- [10] L. Hanu and Unitary team, “Detoxify,” <https://github.com/unitaryai/detoxify>, 2020.
- [11] C. Y. Park, J. Mendelsohn, K. Radhakrishnan, K. Jain, T. Kanakagiri, D. Jurgens, and Y. Tsvetkov, “Detecting community sensitive norm violations in online conversations,” in *Findings of EMNLP*, 2021.
- [12] J. He, J. May, and K. Lerman, “Cpl-novid: Context-aware prompt-based learning for norm violation detection in online communities,” in *Proceedings of ICWSM*, 2024.

- [13] W. Xin, K. Wang, Z. Fu, and L. Zhou, “Let community rules be reflected in online content moderation,” *arXiv preprint arXiv:2408.12035*, 2024.
- [14] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems*, 2013.
- [15] J. Pennington, R. Socher, and C. Manning, “Glove: Global vectors for word representation,” in *Proceedings of EMNLP*, 2014.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT 2019*, 2019, pp. 4171–4186.
- [19] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, “Glue: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of ICLR*, 2019.
- [20] M. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, “Deep contextualized word representations,” in *Proceedings of NAACL*, 2018.
- [21] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving language understanding by generative pre-training,” 2018.
- [22] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text,” in *EMNLP*, 2016.
- [23] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for squad,” in *Proceedings of ACL*, 2018.
- [24] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy, “Race: Large-scale reading comprehension dataset from examinations,” in *EMNLP*, 2017.
- [25] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “Hellaswag: Can a machine really finish your sentence?” in *Proceedings of ACL*, 2019.
- [26] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, “Boolq: Exploring the surprising difficulty of natural yes/no questions,” in *Proceedings of NAACL*, 2019.
- [27] M. Richardson, C. J. Burges, and E. Renshaw, “Mctest: A challenge dataset for the open-domain machine comprehension of text,” in *EMNLP*, 2013.
- [28] E. Chandrasekharan, U. Pavalanathan, A. Srinivasan, A. Glynn, J. Eisenstein, and E. Gilbert, “You can’t stay here: The efficacy of reddit’s 2015 ban examined through hate speech,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 2, no. CSCW, pp. 1–22, 2018.
- [29] J. Lee, J. Bae, J. Yoo, and K. M. Lee, “Grokfast: Accelerated grokking by amplifying slow gradients,” *arXiv preprint arXiv:2405.20233*, 2024.

Implementation Details

.1 Data Augmentation Implementations

The following presents the Python implementation of the three data augmentation strategies described in Section ?? . All three strategies operate on individual dataset instances and return a modified copy without altering the original.

Rule shuffling

Randomly permutes the order of candidate rules within each example:

```
def shuffled_rules(example, only_unsafe):
    example = copy.deepcopy(example)
    rules = list(example['community']['rules'].values())
    random.shuffle(rules)
    example["community"]["rules"] = {
        str(i): v for i, v in enumerate(rules)
    }
    return example
```

Number permutation

Perturbs numerical expressions in the comment text. Short numbers (two digits or fewer) receive a random delta of -1 , 0 , or $+1$. Longer numbers have their digits shuffled while avoiding leading zeros:

```
def permutate_numbers(example):
    example = copy.deepcopy(example)
    text = example["content"]

    def replace_number(m):
        num = m.group(0)
        if len(num) <= 2:
            delta = random.choice([-1, 0, +1])
            return str(max(0, int(num) + delta))
        else:
            digits = list(num)
            random.shuffle(digits)
```

```

        if digits[0] == "0":
            digits[0], digits[-1] = digits[-1], digits[0]
        return "".join(digits)

    example["content"] = re.sub(r"\d+", replace_number, text)
    return example

```

Rule exclusion

Removes a random subset of non-violated rules from the candidate list while always preserving the correct label and at least `min_choices` candidates in total:

```

def exclude_rules(example, min_choices=2):
    example = copy.deepcopy(example)
    rule_texts = list(example["community"]["rules"].values())
    applied = example["applied_rule_text"]
    applied_idx = rule_texts.index(applied)
    candidate_indices = [i for i in range(len(rule_texts))
                        if i != applied_idx]
    max_removable = max(0, len(candidate_indices) - (min_choices-1))
    if max_removable <= 0:
        return example
    num_remove = random.randint(1, max_removable)
    remove_indices = set(random.sample(candidate_indices, num_remove))
    new_rule_texts = [rule_texts[i] for i in range(len(rule_texts))
                    if i not in remove_indices]
    example["community"]["rules"] = {
        str(i): v for i, v in enumerate(new_rule_texts)
    }
    return example

```

Augmentation pipeline

The unified pipeline applies the selected strategies to each training example and removes duplicates using a fingerprint based on comment content and rule order. The `n_replicas_per_strategy` parameter controls how many augmented copies are generated per example per active strategy:

```

def augment_qa(data, do_shuffle=True,
              do_permutate_numbers=False,
              do_exclude_rules=False,
              n_replicas_per_strategy=1):
    augmented = []
    seen = set()

    def fingerprint(ex):
        content = ex.get("content", "")
        rules_vals = tuple(ex["community"]["rules"].values())
        return (content, rules_vals)

```



```

for entry in data:
    seen.add(fingerprint(entry))
    for _ in range(n_replicas_per_strategy):
        if do_shuffle:
            aug = shuffled_rules(entry, only_unsafe=True)
            fp = fingerprint(aug)
            if fp not in seen:
                augmented.append(aug)
                seen.add(fp)
        if do_permutate_numbers:
            aug = permutate_numbers(entry)
            fp = fingerprint(aug)
            if fp not in seen:
                augmented.append(aug)
                seen.add(fp)
        if do_exclude_rules:
            aug = exclude_rules(entry)
            fp = fingerprint(aug)
            if fp not in seen:
                augmented.append(aug)
                seen.add(fp)
return data + augmented

```

.2 McNemar's Test Implementation

The following code was used to perform McNemar's test comparing the predictions of the multiple-choice and extractive question answering models across all three evaluation splits.

```

import json
import numpy as np
from scipy.stats import chi2

def mcnemar_test(multqa_preds, multqa_labels,
                 bertqa_preds, bertqa_labels):
    multqa_correct = (np.array(multqa_preds) ==
                      np.array(multqa_labels))
    bertqa_correct = (np.array(bertqa_preds) ==
                      np.array(bertqa_labels))

    multqa_only = np.sum(multqa_correct & ~bertqa_correct)
    bertqa_only = np.sum(~multqa_correct & bertqa_correct)

    statistic = (abs(multqa_only - bertqa_only) - 1) ** 2 / \
                (multqa_only + bertqa_only)
    p_value = 1 - chi2.cdf(statistic, df=1)

    return statistic, p_value

```

```

splits = {
    'stratified': (
        'multqa_preds_test_stratified.json',
        'bertqa_test_stratified_epoch_8.json'
    ),
    'unseen_rules': (
        'multqa_preds_test_n_rules_out.json',
        'bertqa_test_n_rules_out_epoch_2.json'
    ),
    'unseen_communities': (
        'multqa_preds_test_n_communities_out.json',
        'bertqa_test_n_communities_out_epoch_1.json'
    )
}

for split_name, (multqa_file, bertqa_file) in splits.items():
    multqa = json.load(open(multqa_file))
    bertqa = json.load(open(bertqa_file))
    stat, p = mcnemar_test(
        multqa['predictions'], multqa['labels'],
        bertqa['results_by_start']['predictions'],
        bertqa['results_by_start']['labels']
    )
    print(f"{split_name}: stat={stat:.4f}, p={p:.4f}")

```

.3 Baseline Implementations

The following presents the implementation of the four baseline strategies evaluated in this work. All baselines were implemented using the scikit-learn library.

```

def always_predict_0(X_train, y_train, X_tests):
    dummy_clf = DummyClassifier(strategy="most_frequent")
    dummy_clf.fit(X_train, y_train)
    return [dummy_clf.predict(X_test) for X_test in X_tests]

def random_predict(X_train, y_train, X_tests):
    dummy_clf = DummyClassifier(strategy="stratified")
    dummy_clf.fit(X_train, y_train)
    return [dummy_clf.predict(X_test) for X_test in X_tests]

def predict_nb(X_train, y_train, X_tests):
    pipeline = Pipeline([
        ("vect", TfidfVectorizer(max_features=1000)),
        ("clf", ComplementNB()),
    ])
    pipeline.fit(X_train.content, y_train)
    return [pipeline.predict(X_test.content) for X_test in X_tests]

def predict_svm(X_train, y_train, X_tests):
    if y_train.nunique() < 2:

```

```
        return [np.full(len(X_test), y_train.unique()[0])
                for X_test in X_tests]
pipeline = Pipeline([
    ("vect", HashingVectorizer(n_features=1000)),
    ("clf", LinearSVC()),
])
pipeline.fit(X_train.content, y_train)
return [pipeline.predict(X_test.content) for X_test in X_tests]
```